Cornet Lilian
Cros Léa
Segura-Martinez Hina

Glass Garden

Un jardin en vue du dessus. Au centre, 5 miroirs. Chacun téléporte vers une version alternée du même jardin : nuit, rêve, automne, apocalypse, magie. Dans chaque monde, une étoile à récupérer en résolvant le défi du monde. Puis retour au jardin par le miroir, et ce miroir disparaît définitivement. Quand toutes les étoiles sont ramassées et que tu es dans le jardin : victoire. Un chronomètre tourne en continu, c'est le score.

Le concept

Un jardin paisible en vue du dessus. En son centre, cinq miroirs. Chacun ouvre sur une version alternée du même jardin : la nuit, un rêve, un automne, un monde en ruines, un jardin magique. Chaque monde dissimule une étoile, protégée par une mécanique qui lui est propre. Le principe narratif est *un même lieu, plusieurs réalités* — inspiré d'*Alice au pays des merveilles*.

L'objectif du joueur

1. Entrer dans un miroir, dans l'ordre que l'on veut.
2. Comprendre la mécanique du monde et relever son défi.
3. Récupérer l'étoile, puis revenir au jardin par le miroir.
4. Recommencer jusqu'à avoir toutes les étoiles.

Victoire : toutes les étoiles ramassées et le joueur de retour dans le jardin. La condition est vérifiée à chaque image, mais uniquement dans le monde normal :

```
if (this.isNormalWorld() && areAllStarsCollected()) {
  this.winGame();
}
```

Échec : il n'y a pas de mort définitive. Un contact avec un ennemi renvoie le joueur au point de départ du monde ; une explosion de bombe fait recommencer le monde entier. Dans les deux cas, le chronomètre ne s'arrête jamais : l'échec coûte du temps, pas une vie.

Le score

Un chronomètre global tourne en continu du début à la fin de la partie. Le temps final est le score. Il survit aux changements de monde, aux échecs et aux redémarrages de niveau.

Monde	Carte	Mécanique codée	Contenu Tiled
Jardin	hub_test.json	— (plaque tournante)	5 miroirs, 1 cheminée
1 · Nuit	monde_1.json	récupérer une étoile	
2 · Rêve	monde_2.json	Combattre des ennemis	4 ennemis minimum
3 · Automne	monde_3.json	Énigme d'un PNJ	1 PNJ
4 · Apocalypse	monde_4.json	Tir sur caisses	48 caisses, dont seulement 1 cache l'étoile et une proportion de 25% cache des bombes
5 · Magie	monde_5.json	Éclater des bulles pour un PNJ	7 bulles, 1 PNJ

Le jeu compte 9 scènes enregistrées (`PreloadScene`, `MenuScene`, `HubScene`, `World1` à `World5`, `UIScene`) et une classe mère non enregistrée, `WorldScene`.

Architecture des fichiers

src/
├── **config.js** *la configuration Phaser*
├── **scenes/** *un écran = une scène Phaser*
│ ├── **PreloadScene** *chargement*
│ ├── **MenuScene** *titre (inutilisée pour l'instant)*
│ ├── **WorldScene** ★ *la CLASSE MÈRE de tous les mondes (674 lignes)*
│ ├── **HubScene** *13 lignes*
│ ├── **World1…5Scene** *14 lignes chacune*
│ └── **UIScene** *chrono + compteurs, EN PARALLÈLE (launch)*
├── **systems/** *17 modules de logique, sans écran propre*
├── **data/** *dialogues.js — les textes, hors du code*
└── **utils/**
├── **constants.js** ★ *TOUTES les valeurs du jeu (419 lignes)*
├── **helpers.js** *formatTime, pickVariant*
└── **palette.js** *calcul de couleurs (en réserve)*

Le choix `scenes/` vs `systems/` :

- Une **scène** a un écran et un cycle de vie Phaser. Elle est **détruite et recréée**.
- Un **système** est un module sans écran, qui rend un service. Il est chargé **une seule fois pour toute la partie**.

WorldScene

Le fichier est le moule de tous les mondes avec la carte, joueur, collisions, étoiles, miroirs. Les 6 mondes en héritent les instructions et ne changent que la carte.
Mise en place du système d'héritage avec `WorldScene,` qui est la classe mère (extends). Elle ne connaît que des actions mais aucune touche, les touches sont définies dans InputManager.js (cela permet de centraliser leur gestion).

```
9   export default class HubScene extends WorldScene {
10    constructor() {
11      super(SCENES.HUB, WORLD_MAPS.hub.key, WORLD_MAPS.hub.path);
12    }
13  }
14
```

`extends WorldScene` hérite de `preload`, `create`, `update` et des 20 méthodes. Le constructeur dit juste « voici mon nom de scène et ma carte » — et le constructeur parent les range dans `this.mapKey` / `this.mapPath`.

```
56    constructor(sceneKey, mapKey, mapPath) {
57      super(sceneKey);
58      this.mapKey = mapKey;
59      this.mapPath = mapPath;
60    }
```

UIScene

Pour ce fichier, nous avons choisi le launch : l'UI est donc lancée une seule fois et **jamais détruite**, alors que les mondes sont détruits puis reconstruits après chaque passage à travers un miroir. Le launch empêche le chrono de clignoter ou de se réinitialiser pendant les téléportations.

Creation du timer :

```
22    this.timerText = this.add
23      .text(this.scale.width - UI_MARGIN, UI_MARGIN, formatTime(0), {
24        fontFamily: FONT_FAMILY,
25        fontSize: FONT_SIZE.TIMER,
26        color: COLORS.TEXT,
27      })
28      .setOrigin(1, 0);
```

Mise à jour en 1 ligne : this.timerText.setText(formatTime(timer.elapsed()));

Le chronomètre ne compte d'ailleurs pas les secondes : il mémorise l'heure de départ et soustrait, ce qui le rend exact indépendamment de la fluidité d'affichage :

```
elapsed() {
  if (!this.running) return this.accumulated;
  return this.accumulated + (performance.now() - this.startTime);
}
```

Modèles de mobilité

Le jeu comporte **six modèles de déplacement distincts**, chacun répondant à un besoin différent.

Marche du joueur
Le déplacement n'est pas appliqué directement : on fixe une vitesse et c'est le moteur physique qui déplace et arrête contre les obstacles.

```
this.player.setVelocity(x * PLAYER.SPEED, y * PLAYER.SPEED);
```

```
209   export const PLAYER = {
210     SPEED: 300, // pixels par seconde
```

La normalisation diagonale est centralisée dans l'`InputManager` :

```
if (x !== 0 && y !== 0) { x *= Math.SQRT1_2; y *= Math.SQRT1_2; }
```

Sans cette division par √2, un déplacement diagonal serait environ **1,41 fois plus rapide** qu'un déplacement en ligne droite.

Le dash
L'intention est de partir très vite, freiner progressivement, et parcourir **toujours exactement 135 pixels**, quelle que soit la courbe de freinage choisie.

```
const progress   = (now - this.startedAt + this.scene.game.loop.delta / 2) / DASH.DURATION_MS;
const startSpeed = (DASH.DISTANCE * (DASH.EASE_POWER + 1)) / (DASH.DURATION_MS / 1000);
const speed      = startSpeed * Math.max(1 - progress, 0) ** DASH.EASE_POWER;
```

Le raisonnement :

- la vitesse suit la courbe `v(t) = v₀ × (1 − t)^p` ;
- la distance parcourue est **l'aire sous cette courbe**, soit `v₀ × durée / (p + 1)` ;
- on veut que cette distance vaille `DISTANCE`, donc on **inverse la formule** pour en déduire `v₀`.

Régler `EASE_POWER` change ainsi le *ressenti* du dash (sec ou souple) sans jamais changer sa portée.

Le `+ delta / 2` évalue la courbe au **milieu** de l'image à venir plutôt qu'à son début : comme la vitesse décroît, prendre sa valeur initiale surestimerait systématiquement le déplacement et le dash dépasserait sa distance.

Le dash accorde aussi une **invincibilité** pendant toute sa durée, ce qui en fait un outil défensif autant qu'un outil de déplacement.

Poursuite des ennemis
Le comportement est une machine à quatre états :

- idle → il flâne autour de son point de départ
- chase → il voit le joueur (distance < sight ET aucun mur entre eux) : il fonce
- search → il l'a perdu : il va au dernier endroit vu et y attend
- return → il rentre à son point de départ, puis repasse en idle

Ennemi	Points de vie	Vitesse	Portée de vue
Faucheur	3	220 px/s	280 px
Capuche (fantôme à tentacules)	2	160 px/s	200 px

Les deux sont **plus lents que le joueur** (300 px/s) : fuir est toujours possible, choix de game design autant que technique.

Deux raffinements notables :

- Hystérésis (LOSE_SIGHT_FACTOR: 1.3) : un ennemi lancé ne lâche sa cible qu'au-delà de 1,3 × sa portée de vue. Sans cette marge, un joueur pile à la limite ferait clignoter l'ennemi entre poursuite et abandon à chaque image.
- Abandon de sécurité (SEARCH.MAX_MS: 3500) : si l'ennemi reste bloqué, il renonce au bout de 3,5 secondes au lieu de s'acharner contre un obstacle.

Les ennemis contournent les obstacles, et sont bloqués par les murs, l'eau, les caisses, les PNJ, les miroirs et entre eux.

Rebond balistique
Lorsqu'une étoile est libérée, elle **jaillit et rebondit** comme un objet réel. Deux grandeurs sont suivies séparément : la **position au sol**, qui avance en ligne droite, et la **hauteur**, qui suit une parabole. L'objet est dessiné à `sol - hauteur`, avec une ombre portée qui rétrécit à mesure qu'il monte.

Les rebonds sont **calculés à l'avance** : chaque impact repart à 55 % de la vitesse d'arrivée.

```
const impact   = Math.sqrt(speed * speed + 2 * GRAVITY * height);  // vitesse à l'impact
const duration = (speed + impact) / GRAVITY;
speed = impact * RESTITUTION;
```

La **direction** est choisie parmi 24 directions réparties sur le cercle, testées dans un ordre aléatoire ; on retient la première dont **tout le trajet est libre**. Si aucune ne convient, le trajet est raccourci à 60 %, puis 30 %, puis l'objet retombe sur place. C'est ce qui garantit qu'une étoile ne peut jamais atterrir dans un mur et devenir inaccessible.

Le même moteur sert aux **bombes**, avec une portée divisée par deux.

Flottement des bulles
Les bulles ne sont pas physiques : leur dérive est purement calculée.

```
bubble.setPosition(
  originX + Math.sin(seconds * SPEED_X + phase) * AMPLITUDE_X,
  originY + Math.sin(seconds * SPEED_Y + phase * 1.7) * AMPLITUDE_Y,
);
```

Deux oscillations de **vitesses différentes et non proportionnelles** (1,3 et 2,1) composent une petite boucle irrégulière : l'œil ne perçoit aucune répétition. La `phase` tirée au hasard par bulle évite qu'elles ne se balancent toutes à l'unisson.

Projectiles
Les quatre directions sont décrites une fois pour toutes, vecteur et angle de rotation du sprite :

```
export const DIRECTION_VECTORS = {
  right: { x:  1, y:  0, angle: 0 },
  down:  { x:  0, y:  1, angle: Math.PI / 2 },
  left:  { x: -1, y:  0, angle: Math.PI },
  up:    { x:  0, y: -1, angle: -Math.PI / 2 },
};
```

Tirer revient alors à lire la table, sans aucun `if` par direction. Chaque balle s'auto-détruit après 1,2 seconde : même sortie de l'écran, elle ne vole jamais indéfiniment.

Mécaniques de jeu

Les miroirs
Un miroir est un objet Tiled de classe mirror dont le nom est le monde de destination. La table WORLDS traduit ce nom en scène : const targetScene = WORLDS[nearMirror.world];.

Un miroir disparaît définitivement une fois son monde terminé — mais seulement à une condition :

```
export function shouldMirrorDisappear(world) {
  return isWorldStarCollected(world);
}
```

Si le joueur ressort d'un monde **sans** son étoile, le miroir reste en place et il peut y retourner. Sans cette condition, une sortie prématurée rendrait la partie impossible à finir.

Les étoiles

Toutes les mécaniques reposent sur le même principe : l'étoile existe dès le chargement mais elle est désactivée, et c'est la réussite du défi qui la révèle.

```
if (this.crates.getChildren().some((crate) => crate.hasStar)) {
  this.stars.forEach((star) => star.disableBody(true, true));
}
```

Déclencheur	Méthode appelée	Effet visuel
Énigme résolue	revealWorldStar(npc)	l'étoile saute de la tête du personnage et rebondit
Caisse `hasStar` brisée	onStarCrateBroken(crate)	l'étoile jaillit de la caisse et rebondit
Dernière bulle éclatée	revealStarAtCenter(true)	l'étoile apparaît au centre en grossissant

Le tir et le système de cibles

Le système de tir ne connaît ni les caisses ni les ennemis. Il ne connaît que des « cibles » : n'importe quel objet physique disposant éventuellement de hitPoints et d'une fonction onHit.

```
target.hitPoints = (target.hitPoints ?? 1) - 1;
target.onHit?.(target);
if (target.hitPoints <= 0) target.destroy();
```

Les caisses

Une caisse est un objet Tiled de classe `crate`, avec trois propriétés facultatives : `hitPoints` (nombre de tirs), `hasStar` (elle cache l'étoile), `hasBomb` (elle cache une bombe).

Une bombe libérée **jaillit, rebondit, se pose, clignote 300 ms, puis explose** dans un rayon de 80 pixels :

- un joueur dans le rayon → **le monde recommence** (l'invincibilité et le dash protègent) ;
- un ennemi dans le rayon → il meurt ;

- une caisse dans le rayon → elle casse — **et si elle cachait une bombe, la chaîne se propage**.

C'est la mécanique la plus riche du jeu : une seule balle bien placée peut déclencher une cascade d'explosions, avec un vrai risque pour le joueur.

Les bulles
Trois tailles (large, medium, small, réglées par une propriété Tiled size), et éclater une grosse bulle en engendre de plus petites :

```
SPLIT: {
  large:  { INTO: 'medium', COUNT: 4 },
  medium: { INTO: 'small',  COUNT: 2, DISTANCE: 48 },
},
```

Les grosses bulles posées dans la carte engendrent donc une arborescence de plusieurs dizaines de bulles, à partir de trois objets seulement dans l'éditeur.

L'enigme
Le PNJ du monde 3 prononce ses répliques (effet machine à écrire, 28 ms par lettre), puis pose une question à quatre réponses en deux colonnes, que le joueur parcourt au joystick. Une mauvaise réponse autorise un nouvel essai.

Le DialogueSystem ignore totalement ce qu'est une étoile. Il reçoit une fonction de récompense et se contente de l'appeler :

```
this.dialogue.open(nearNpc.npcName, DIALOGUES[nearNpc.npcName], () =>
this.revealWorldStar(nearNpc));
```

Les textes vivent dans src/data/dialogues.js — jamais dans le code.

Échec, respawn et redémarrage

Événement	**Conséquence**
Contact avec un ennemi	retour au point de départ du monde, les ennemis vivants reprennent leur place, **1,5 s d'invincibilité** avec clignotement et éclair rouge
Explosion de bombe sur un joueur	**le monde entier recommence** (caisses, ennemis, bombes réinitialisés)

Un ennemi déjà tué ne revient pas et le chronomètre continue de tourner dans tous les cas.

Le mode deux joueurs
Le second joueur dispose de ses propres touches sur la borne (second joystick + boutons R/T/Y) et réutilise exactement les mêmes systèmes que le joueur 1 :

```
p2.input2 = new InputManager(scene, 2);
p2.dash   = new Dash(scene, p2);
```

Il apparaît **à côté du joueur 1**, à la première position libre testée dans les quatre directions — et non à un point fixe qui pourrait se trouver dans un mur. Les ennemis le poursuivent et le touchent comme le joueur 1, et il peut éclater les bulles.